

Foundations of Artificial Intelligence

M. Helmert
C. Büchner

University of Basel
Spring Term 2026

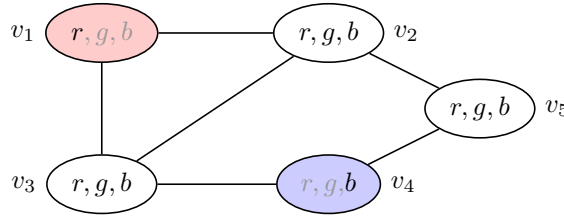
Exercise Sheet 7

Due: April 19, 2026

Please note the submission rules at the end of this exercise sheet. Non-adherence to these rules may lead to your submission not being marked.

Exercise 7.1 (1+1 marks)

Consider the following graph coloring problem:



Formally, it is given as the constraint network $\mathcal{C} = \langle V, \text{dom}, (R_{xy}) \rangle$ with

$$\begin{aligned} V &= \{v_1, v_2, v_3, v_4, v_5\} \\ \text{dom}(v) &= \{r, g, b\} \text{ for all } v \in V \\ R_{xy} &= \{\langle r, g \rangle, \langle r, b \rangle, \langle g, r \rangle, \langle g, b \rangle, \langle b, r \rangle, \langle b, g \rangle\} \\ &\text{for all } \langle x, y \rangle \in \{\langle v_1, v_2 \rangle, \langle v_1, v_3 \rangle, \langle v_2, v_3 \rangle, \langle v_2, v_4 \rangle, \langle v_3, v_4 \rangle, \langle v_4, v_5 \rangle\} \end{aligned}$$

Now consider the partial assignment $\alpha = \{v_1 \mapsto r, v_4 \mapsto b\}$ as indicated in the illustration and assume this leads to the reduced domains $\text{dom}(v_1) = \{r\}$ and $\text{dom}(v_4) = \{b\}$.

- Apply the algorithm AC-3 (chapter D5) on $\mathcal{C}$ to compute an equivalent arc consistent network $\mathcal{C}'$. Select the variables u and v in each iteration of the while loop such that the domain of u changes in the call to **revise**($\mathcal{C}, u, v$). For each iteration, specify the call to **revise** and the updated domain $\text{dom}(u)$. Specify the full constraint network $\mathcal{C}''$ you obtained.
- Apply the algorithm PC-2 (chapter D6) on $\mathcal{C}$ to compute an equivalent path consistent network $\mathcal{C}''$. Select the variables u, v , and w in each iteration of the while loop such that R_{uv} changes in the call to **revise-3**($\mathcal{C}, u, v, w$). For each iteration, specify the call to **revise-3** and the updated constraint R_{uv} . Specify the full constraint network $\mathcal{C}''$ you obtained.

Note that you do *not* have to provide the elements that are inserted into the queue in either case above, and you may stop the algorithm as soon as there are no u, v or u, v, w such that $\text{dom}(u)$ or R_{uv} changes, respectively.

Exercise 7.2 (1+2 marks)

Consider the constraint network $\mathcal{C} = \langle V, \text{dom}, (R_{uv}) \rangle$ with $V = \{A, B, C, D, E, F, G\}$, $\text{dom}(v) = \{1, \dots, 10\}$ for all $v \in V$ and $R_{uv} = \text{dom}(u) \times \text{dom}(v)$ for all pairs $u, v \in V$ except for:

$$\begin{aligned} R_{AC} &= \{\langle x, y \rangle \in \text{dom}(A) \times \text{dom}(C) \mid x \leq y\} \\ R_{BG} &= \{\langle x, y \rangle \in \text{dom}(B) \times \text{dom}(G) \mid x < y\} \\ R_{CE} &= \{\langle x, y \rangle \in \text{dom}(C) \times \text{dom}(E) \mid 2x < y\} \\ R_{CG} &= \{\langle x, y \rangle \in \text{dom}(C) \times \text{dom}(G) \mid |x - y| > 3\} \\ R_{DF} &= \{\langle x, y \rangle \in \text{dom}(D) \times \text{dom}(F) \mid |y - x^2| \leq 1\} \\ R_{FG} &= \{\langle x, y \rangle \in \text{dom}(F) \times \text{dom}(G) \mid |x - y| \leq 2\} \end{aligned}$$

- (a) Draw the constraint graph of $\mathcal{C}$.
- (b) Apply the algorithm for trees as constraint graphs (introduced in chapter D7) to solve $\mathcal{C}$. Provide the following:
 - the ordered tree, rooted at A ,
 - a variable order that is given by the ordered tree,
 - the sequence of calls of the `revise` method and the resulting variable domains, and
 - the result of a backtracking search (using your variable order, and forward checking after each variable assignment) with a value order that prefers smaller values. It suffices to provide the final variable assignments (it is, in particular, not necessary to provide the complete search tree).

Exercise 7.3 (2+1 marks)

Consider the constraint network that is given by the graph coloring problem of a graph $G = \langle V, E \rangle$. The set of vertices V contains a vertex for each Swiss canton, and E is such that two vertices v and v' are connected iff the cantons v and v' share a border. A description of G can be downloaded from ADAM and you can visualize it with graphviz:

```
dot -T pdf -o cantons.pdf cantons.dot
```

- (a) Provide a cutset $V' \subseteq V$ for G that is as small as possible (it is not necessary to provide an explanation how you have found V'). As a reminder, a cutset of a graph is defined as a set of vertices that is such that the induced subgraph that is obtained by removing these vertices results in an acyclic graph.
Note: You get 2 marks for your solution if your cutset is optimal, 1 mark if your cutset contains exactly one more vertex than an optimal cutset and 0 marks otherwise.
- (b) Assume we are interested in coloring G with 4 colors. Provide a worst-case runtime estimate of the algorithm based on cutset conditioning if your cutset from the first part of this exercise is used (i.e., compute an upper bound for the number of considered assignments). Compare your result to the estimated runtime if no cutset is used.

Exercise 7.4 (2 marks)

Let $\mathcal{C}$ be a solvable constraint network with an acyclic constraint graph. Show that the application of the algorithm for trees as constraint graphs (introduced in chapter D7) leads to a solution for $\mathcal{C}$ without needing to backtrack.

Hint: First use structural induction to show that after step 3 (i.e., after calling the revise procedure for parent-child pairs), the domain of every node v contains only values for which a consistent assignment to all variables in the subtree rooted at v can be found. Then use this to show that backtracking with forward checking will always pick a value that can be extended to a solution. Also consider that finding a solution requires that no domain ever becomes empty.

Submission rules:

- Submit one solution in groups of up to two students, stating the names of all group members in the submission. All group members are equally responsible for all submitted exercises.
- Create a single A4 PDF file for all non-programming exercises. Solutions in L^AT_EX are preferred, but clearly legible scans in A4 with reasonable contrast and file size are accepted as well.
- In case there are programming exercises, upload a ZIP file (ending .zip, .tar.gz or .tgz; *not* .rar or anything else) including your PDF and those code textfiles required by the exercise. Put your names in a comment on top of each code file. Make sure your code compiles and test it. Code that does not compile or which we cannot successfully execute will not be graded.

- Include a statement on scientific integrity at the end of your submission (see below).

Statement on scientific integrity:

- All submitted solutions must be the product of your own work.
- If your written solutions were produced using any help (e.g., from peers or a large language model), list in what way these people or tools contributed to the solutions.
- We do not mark submissions that lack a statement on scientific integrity.
- Examples:
 - “We solved all exercises on our own.”
 - “We used ChatGPT to explain exercise 7 which we did not understand.”
 - “We had no time to solve this exercise sheet and copied all solutions from our friends.”
(If this is the case, thanks for your honesty but this is plagiarism, results in a warning and no marks. It is preferable to submit no solution at all.)